

Guide Complet de Révision

Examens Java EE (J2EE) - 4IIR

Synthèse des connaissances requises

Support de Révision

D'après les examens 2022-2023, 2023-2024, 2024-2025

14 juin 2026

Table des matières

1	Introduction - Vue d'ensemble des examens	5
1.1	Matières couvertes dans les examens	5
1.2	Fréquence des questions JEE dans les examens	6
2	Java EE / Jakarta EE - Questions de cours	7
2.1	Définition du standard Java EE	7
2.2	Rôle du conteneur web (Tomcat)	7
2.3	Serveurs d'application Java EE	8
2.4	Services fournis par un serveur d'application	8
3	Servlets - Questions détaillées	9
3.1	Cycle de vie d'une Servlet	9
3.2	Méthodes HTTP - GET vs POST vs PUT	10
3.3	Configuration d'une Servlet	10
3.3.1	Façon 1 : Annotation (recommandée)	10
3.3.2	Façon 2 : Fichier web.xml	11
3.4	forward() vs sendRedirect()	11
4	JSP (JavaServer Pages) - Questions de cours	13
4.1	Différence entre Servlet et JSP	13
4.2	Objets implicites JSP	14
4.3	Rôle du JSTL	14
5	Architecture MVC - Questions détaillées	15
5.1	Problème résolu par MVC	15
5.2	Rôle de la couche Service	16
5.3	Rôle de la couche DAO	16
6	JDBC - Questions de cours	17
6.1	Différence Statement vs PreparedStatement	17
6.2	Les 4 classes importantes de JDBC	17
7	JPA - Questions détaillées	19
7.1	Points forts de JPA par rapport à JDBC	19
7.2	Différence JPA vs Hibernate	19
7.3	Rôle du fichier persistence.xml	20
7.4	Rôle de l'EntityManager	20
7.5	Stratégies d'héritage JPA (JOINED)	20
7.6	Relations JPA (Many-to-One / One-to-Many)	20

8 Spring Framework - Questions détaillées	21
8.1 Spring Boot Starters	21
8.2 Rôle de l'annotation @Query	21
8.3 Rôle de l'annotation @Autowired	21
9 Schéma d'architecture 3 couches à compléter	23
10 Codes de statut HTTP à connaître	25
11 Exercices types avec corrigés	27
11.1 Exercice 1 : Création d'une Servlet complète	27
11.2 Exercice 2 : Configuration web.xml	28
12 Synthèse - Ce qu'il faut absolument retenir	31
12.1 Tableau récapitulatif des différences essentielles	31
12.2 Annotations à connaître par cœur	32
12.3 Checklist de révision finale	33

Chapitre 1

Introduction - Vue d'ensemble des examens

1.1 Matières couvertes dans les examens

D'après l'analyse de tous les examens fournis, voici les matières qui tombent régulièrement à l'examen de 4ème année IIR :

Liste des matières d'examen

- **J2EE / Java EE** - L'objet de ce guide
- Bases de Données Avancées (B.O.O, NoSQL)
- Data Warehouse (ETL, Cubes OLAP, Modélisation)
- Génie Logiciel (UML, Agile, Design Patterns)
- Sécurité des Réseaux (ACL, VPN, Cryptographie)
- Administration Windows / Azure
- Virtualisation (Hyperviseurs, VM)
- Développement Mobile (Android)
- Entrepreneuriat / Management
- Anglais / Techniques d'Expression

1.2 Fréquence des questions JEE dans les examens

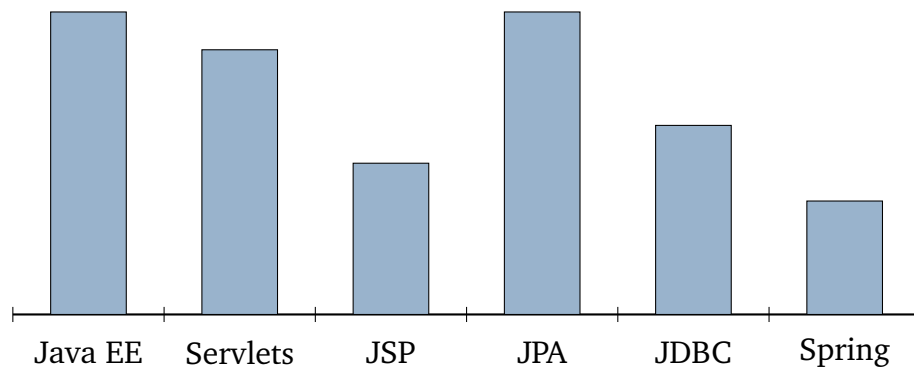


FIGURE 1.1 – Fréquence approximative des questions par thème JEE

Chapitre 2

Java EE / Jakarta EE - Questions de cours

2.1 Définition du standard Java EE

Question type (Examen 2025)

Définir le standard Java EE ? (1 point)

Réponse attendue

Java EE (Java Platform, Enterprise Edition), maintenant appelé **Jakarta EE**, est une plateforme standardisée pour le développement d'applications d'entreprise **distribuées, robustes, sécurisées et évolutives**. Elle fournit :

- Un ensemble de **spécifications** (Servlets, JSP, EJB, JPA, JMS, ...)
- Des **API** pour simplifier le développement
- Un environnement d'exécution via des **conteneurs** (web, EJB)
- Des services intégrés (transactions JTA, nommage JNDI, sécurité, pooling de connexions)

2.2 Rôle du conteneur web (Tomcat)

Question type (Examen 2025)

C'est quoi le rôle du conteneur WEB (exemple Tomcat) dans le standard Java EE ? (1 point)

Réponse attendue

Le conteneur web (comme Apache Tomcat) est responsable de :

1. **Gérer le cycle de vie** des Servlets et JSP (init, service, destroy)
2. **Mapper les URLs** vers les Servlets appropriées
3. **Traiter les requêtes HTTP** et générer les réponses
4. **Gérer les sessions** utilisateur (HttpSession)
5. **Fournir des services de sécurité** (authentification, autorisation)
6. **Gérer les pools de connexions** vers les bases de données

2.3 Serveurs d'application Java EE

Question type (Examen 2025)

Citer deux exemples de serveurs d'application Java EE ? (2 points)

Réponse attendue

- **GlassFish** - Référence Open Source (Oracle/Eclipse)
- **WildFly (ex-JBoss)** - Serveur Open Source de Red Hat
- **WebLogic** - Serveur commercial Oracle
- **WebSphere** - Serveur commercial IBM
- **TomEE** - Apache Tomcat + extensions Java EE
- **Payara** - Fork de GlassFish

2.4 Services fournis par un serveur d'application

Service	Description
Service de nommage (JNDI)	Localisation des ressources (DataSources, EJB) par nom logique
Service de transactions (JTA)	Gestion des transactions distribuées (commit/rollback)
Service de sécurité	Authentification, autorisation, chiffrement
Service de déploiement	Gestion du cycle de vie des applications (WAR, EAR, JAR)
Pooling de connexions	Réutilisation des connexions BDD pour les performances
Service de messagerie (JMS)	Communication asynchrone entre composants

TABLE 2.1 – Services fournis par un serveur d'application Java EE

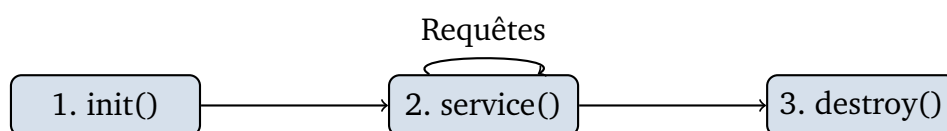
Chapitre 3

Servlets - Questions détaillées

3.1 Cycle de vie d'une Servlet

Question type (Examen 2025)

Expliquer le cycle de vie de la Servlet? (2 points)



Une seule instance, multiples threads

FIGURE 3.1 – Cycle de vie d'une Servlet

Réponse détaillée

1. **init()** :

- Appelée **UNE SEULE FOIS** lors du chargement de la Servlet
- Utilisée pour initialiser les ressources (connexions BDD, fichiers)
- Peut être appelée au démarrage ou à la première requête selon configuration

2. **service()** / **doGet()** / **doPost()** :

- Appelée à **CHAQUE REQUÊTE** du client
- La méthode `service()` lit le type de requête (GET/POST/PUT/DELETE)
- Délègue à la méthode appropriée (`doGet`, `doPost`, etc.)
- S'exécute dans un thread séparé pour chaque requête

3. **destroy()** :

- Appelée **UNE SEULE FOIS** lors de l'arrêt de l'application
- Utilisée pour libérer les ressources (fermer BDD, fichiers)

3.2 Méthodes HTTP - GET vs POST vs PUT

Question type (Examen 2025)

C'est quoi la différence entre les méthodes POST et PUT dans HTTP? (1 point)

Caractéristique PUT	GET	POST
But	Récupérer des données	Créer une ressource
Remplacer complètement une ressource		
Paramètres Dans le corps	Dans l'URL	Dans le corps
Taille limite Non	Oui (2048 carac)	Non
Sécurité Plus sécurisé	Non sécurisé	Plus sécurisé
Idempotence Oui	Oui	Non
Cache Non	Oui	Non
Bookmark Impossible	Possible	Impossible

TABLE 3.1 – Comparaison des méthodes HTTP

Réponse sur POST vs PUT

- **POST** : Utilisée pour **créer** une nouvelle ressource. La même requête exécutée plusieurs fois peut créer plusieurs ressources (non idempotent).
- **PUT** : Utilisée pour **remplacer** complètement une ressource existante à une URL spécifique. La même requête exécutée plusieurs fois produit le même résultat (idempotent).

3.3 Configuration d'une Servlet

Question type (Examen 2024)

Citer les deux façons pour configurer un servlet nommé HomeServlet du package web dans un projet Java EE? (2 points)

3.3.1 Façon 1 : Annotation (recommandée)

```
1 package web;
2 import javax.servlet.annotation.WebServlet;
3 import javax.servlet.http.HttpServlet;
4
5 @WebServlet(name = "HomeServlet", urlPatterns = {"/home", "/accueil"})
6 public class HomeServlet extends HttpServlet {
7     // ...
8 }
```

3.3.2 Façon 2 : Fichier web.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3     version="3.1">
4
5     <!-- Declaration de la Servlet -->
6     <servlet>
7         <servlet-name>HomeServlet</servlet-name>
8         <servlet-class>web.HomeServlet</servlet-class>
9     </servlet>
10
11     <!-- Mapping URL -->
12     <servlet-mapping>
13         <servlet-name>HomeServlet</servlet-name>
14         <url-pattern>/home</url-pattern>
15         <url-pattern>/accueil</url-pattern>
16     </servlet-mapping>
17
18 </web-app>
```

3.4 forward() vs sendRedirect()

Question type (Examen 2024)

Quelle est la différence entre l'utilisation de forward() et sendRedirect() ? (1.5 points)

forward()	sendRedirect()
S'exécute côté serveur	S'exécute côté client (navigateur)
Une seule requête HTTP	Deux requêtes HTTP
Les attributs sont conservés	Les attributs sont perdus
URL ne change pas dans navigateur	URL change dans navigateur
Limitée au même contexte web	Peut rediriger vers URL externe
Plus performante	Moins performante
Utilise <code>RequestDispatcher</code>	Utilise <code>HttpServletResponse</code>

TABLE 3.2 – Différence entre `forward()` et `sendRedirect()`

```
1 // forward() - redirection interne
2 RequestDispatcher rd = request.getRequestDispatcher("/page.jsp");
3 rd.forward(request, response);
4
5 // sendRedirect() - redirection externe ou interne
6 response.sendRedirect("http://www.exemple.com");
7 response.sendRedirect(request.getContextPath() + "/accueil");
```

Chapitre 4

JSP (JavaServer Pages) - Questions de cours

4.1 Différence entre Servlet et JSP

Question type (Examen 2024)

Quelle est la différence entre Servlet et JSP ? (1 point)

Réponse attendue

- **Servlet :**
 - Code Java pur
 - Mieux adaptée pour le **contrôle** (Contrôleur dans MVC)
 - Génère du HTML via `out.println()`
 - Plus difficile à maintenir pour le HTML complexe
- **JSP :**
 - Mélange de HTML et Java
 - Mieux adaptée pour la **présentation** (Vue dans MVC) séparation du design et de la logique
 - Convertie en Servlet par le conteneur
 - Plus facile à maintenir pour les développeurs web

4.2 Objets implicites JSP

Objet	Description
request	Requête HTTP (HttpServletRequest)
response	Réponse HTTP (HttpServletResponse)
session	Session utilisateur (HttpSession)
application	Contexte de l'application (ServletContext)
out	Flux de sortie (JspWriter)
pageContext	Contexte de la page
config	Configuration de la Servlet
page	Instance courante de la page
exception	Uniquement dans les pages d'erreur

TABLE 4.1 – Objets implicites en JSP

4.3 Rôle du JSTL

Question type (Examen 2024)

Expliquer le rôle du JSTL ? (0.5 point)

Réponse attendue

JSTL (JSP Standard Tag Library) est une bibliothèque de balises personnalisées qui permet :

- Éviter les **scriptlets Java** dans les pages JSP
- Fournir des tags pour les opérations courantes :
 - `<c:forEach>` - Boucles
 - `<c:if>` - Conditions
 - `<c:set>` - Variables
 - `<c:out>` - Affichage
 - `<c:choose>` - Switch/case
- Tags pour le formatage (`<fmt:formatDate>`, `<fmt:formatNumber>`)
- Tags pour l'internationalisation (`i18n`)
- Requêtes SQL (déconseillé en MVC)

Chapitre 5

Architecture MVC - Questions détaillées

5.1 Problème résolu par MVC

Question type (Examen 2025)

C'est quoi le problème qui est résolu par le MVC ? (1 point)

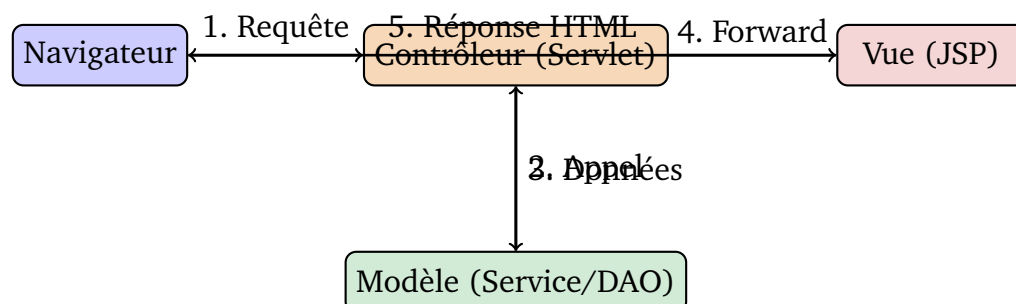


FIGURE 5.1 – Flux MVC dans une application web Java EE

Réponse attendue

Le MVC (Modèle-Vue-Contrôleur) résout le problème du **mélange des responsabilités** dans une application web :

- **Sans MVC** : Code Java, SQL et HTML mélangés (difficile à maintenir, tester et faire évoluer)
- **Avec MVC** : Séparation claire en 3 couches :
 - **Modèle (Model)** : Logique métier et données
 - **Vue (View)** : Présentation (HTML, CSS)
 - **Contrôleur (Controller)** : Orchestration des requêtes

Bénéfices : Meilleure maintenabilité, réutilisabilité, testabilité et séparation des compétences.

5.2 Rôle de la couche Service

Question type (Examen 2025)

Quel est le rôle de la couche service ? (1 point)

Réponse attendue

La couche Service (ou couche métier) a pour rôle :

1. **Contenir la logique métier** (règles de gestion)
2. **Orchestrer** les opérations entre plusieurs DAO
3. **Gérer les transactions** (@Transactional)
4. **Faire le lien** entre le contrôleur et la couche DAO
5. **Valider** les données avant persistance

Exemple :

```
1 @Service
2 public class UserService {
3     @Autowired
4     private UserDao userDao;
5
6     @Transactional
7     public User creerUtilisateur(String nom, String email) {
8         // Logique métier
9         if (nom == null || nom.isEmpty()) {
10             throw new IllegalArgumentException();
11         }
12         return userDao.save(new User(nom, email));
13     }
14 }
```

5.3 Rôle de la couche DAO

Question type (Examen 2024)

Il est recommandé de scinder une application en trois couches, présentation, service et DAO. Expliquer pourquoi ? (0.5 point)

La séparation en trois couches offre :

1. **Séparation des préoccupations** (SoC)
2. **Réutilisabilité** de la couche DAO pour différents services
3. **Testabilité** indépendante de chaque couche
4. **Flexibilité** : possibilité de changer de BDD sans impacter la présentation
5. **Maintenabilité** facilitée par le découplage

Chapitre 6

JDBC - Questions de cours

6.1 Différence Statement vs PreparedStatement

```
1 // Statement - NON SECURISE
2 Statement stmt = conn.createStatement();
3 String sql = "SELECT * FROM users WHERE name = '" + userInput + "'";
4 ResultSet rs = stmt.executeQuery(sql); // Injection possible!
5
6 // PreparedStatement - SECURISE
7 PreparedStatement pstmt = conn.prepareStatement(
8     "SELECT * FROM users WHERE name = ?"
9 );
10 pstmt.setString(1, userInput);
11 ResultSet rs = pstmt.executeQuery(); // Secure!
```

6.2 Les 4 classes importantes de JDBC

Question type (Examen 2024)

Citer les 4 classes les plus importantes dans l'API JDBC ? (1 point)

Réponse attendue

1. **DriverManager** : Charge et configure le pilote JDBC, établit la connexion
2. **Connection** : Représente la connexion à la base de données
3. **Statement/PreparedStatement** : Exécute les requêtes SQL
4. **ResultSet** : Parcourt et stocke les résultats d'une requête SELECT

Chapitre 7

JPA - Questions détaillées

7.1 Points forts de JPA par rapport à JDBC

Quels sont les points forts de JPA par rapport à JDBC ? (1 point)

- **Mapping objet-relationnel (ORM)** : Mapping automatique Java ↔ SQL
- **Pas d'écriture SQL manuelle** : Opérations CRUD générées automatiquement
- **Portabilité** : Changez de BDD sans modifier le code
- **Cache de premier niveau** : Améliore les performances
- **Gestion des relations** : @OneToMany, @ManyToOne, etc.
- **JPQL** : Requêtes orientées objet (pas besoin de connaître SQL)
- **Gestion automatique des transactions**
- **Moins de code boilerplate** : Pas de try-catch-finally pour fermer les ressources

7.2 Différence JPA vs Hibernate

- **JPA (Java Persistence API)** :
 - C'est une **spécification** (interface)
 - Définit les standards (annotations, API)
 - Proposée par Sun/Oracle
 - Exemples : @Entity, @Id, EntityManager, JPQL
- **Hibernate** :
 - C'est une **implémentation** de la spécification JPA
 - Framework ORM open source
 - Ajoute des fonctionnalités supplémentaires (criteria API, cache de second niveau)
 - Peut être utilisé avec ou sans JPA

Relation : Hibernate **implémente** JPA. On peut utiliser Hibernate via l'API JPA standard.

7.3 Rôle du fichier `persistence.xml`

Le fichier `persistence.xml` (placé dans `META-INF/`) configure l'unité de persistance :

- **Données de connexion** à la BDD (driver, URL, user, password)
- **Classes d'entités** à gérer
- **Propriétés spécifiques** à l'implémentation (Hibernate : dialecte, ddl-auto, show_sql)
- **Nom de l'unité** de persistance
- **Type de transactions** (`RESOURCE_LOCAL` vs `JTA`)

7.4 Rôle de l'EntityManager

L'EntityManager est l'interface principale pour interagir avec le contexte de persistance :

- **`persist()`** : Insérer une nouvelle entité
- **`merge()`** : Mettre à jour une entité détachée
- **`find()`** : Rechercher par clé primaire
- **`remove()`** : Supprimer une entité
- **`createQuery()`** : Exécuter des requêtes JPQL
- **`getTransaction()`** : Gérer les transactions (hors Java EE)
- **`flush()`** : Synchroniser avec la BDD
- **`clear()`** : Détacher toutes les entités

7.5 Stratégies d'héritage JPA (JOINED)

```

1 @Getter @Setter @NoArgsConstructor @Entity @Inheritance(strategy =
  InheritanceType.JOINED) // ligne (*) completer public class Personne @Id
  @GeneratedValue private Long id
2     private String nom private String prenom
3 @Entity public class Etudiant extends Personne // Ne pas remettre @Id ici
  private String matricule

```

7.6 Relations JPA (Many-to-One / One-to-Many)

```

4 @Entity public class Article @Id @GeneratedValue private Integer id
5 @ManyToOne //
6 ; ligne (*) completer @JoinColumn(name = "categorie_id")private Categorie categorie
7 @Entity public class Categorie @Id @GeneratedValue private Integer id
8 @OneToMany(mappedBy = "categorie") // ; ligne (*) completer private List<Article>
  articles

```

Chapitre 8

Spring Framework - Questions détaillées

8.1 Spring Boot Starters

8.2 Rôle de l'annotation @Query

L

```
1 // Requete JPQL @Query("SELECT u FROM User u WHERE u.email = email") User  
  findByEmail(@Param("email") String email)  
2 // Requete SQL native @Query(value = "SELECT * FROM users WHERE age > age",  
  nativeQuery = true) List<User> findUsersOlderThan(@Param("age") int age)  
3 // Requete avec jointure @Query("SELECT u.nom, COUNT(c) FROM User u JOIN u.commandes  
  c GROUP BY u") List<Object[]> getCommandeCountPerUser()
```

8.3 Rôle de l'annotation @Autowired

L'annotation **@Autowired** (Spring) indique l'injection automatique de dépendances :

- Spring cherche un bean du type requis dans son conteneur IoC
- Si trouvé, l'injecte automatiquement (par constructeur, setter ou champ)
- Évite de créer manuellement les objets avec "new"
- Peut être combinée avec @Qualifier pour spécifier exactement quel bean injecter

```
4 // Injection par champ (recommandee sur constructeur) @Autowired private UserDao  
  userDao  
5 // Injection par constructeur (recommande) @Autowired public UserService(UserDao  
  userDao) this.userDao = userDao  
6 // Injection par setter @Autowired public void setUserDao(UserDao userDao)  
  this.userDao = userDao
```


Chapitre 9

Schéma d'architecture 3 couches à compléter

Architecture 3 couches complète à connaître pour l'examen

À SAVOIR PLACER DANS LE SCHÉMA

- **Couche Présentation** : Servlet (@WebServlet), JSP, ou Controller Spring (@RestController)
- **Couche Service** : Interface Service + Implémentation (@Service)
- **Couche DAO** : Interface DAO + Implémentation (@Repository)
- **Base de données** : MySQL, PostgreSQL, H2, etc.

Chapitre 10

Codes de statut HTTP à connaître

Code	Nom	Signification
2xx - Succès		
200	OK	Requête réussie, réponse envoyée
201	Created	Ressource créée avec succès (POST/PUT)
204	No Content	Succès mais pas de contenu à renvoyer
3xx - Redirection		
301	Moved Permanently	Redirection permanente
302	Found	Redirection temporaire
304	Not Modified	Ressource non modifiée (cache)
4xx - Erreur client		
400	Bad Request	Requête invalide
401	Unauthorized	Authentification requise
403	Forbidden	Accès interdit (authentifié mais pas autorisé)
404	Not Found	Ressource non trouvée
405	Method Not Allowed	Méthode HTTP non supportée par l'URL (tombe souvent à l'examen)
5xx - Erreur serveur		
500	Internal Server Error	Erreur générique du serveur
501	Not Implemented	Fonctionnalité non implémentée
503	Service Unavailable	Serveur indisponible (surcharge)

TABLE 10.1 – Codes de statut HTTP essentiels

Chapitre 11

Exercices types avec corrigés

11.1 Exercice 1 : Création d'une Servlet complète

Exercice type

Créez une Servlet qui :

1. Affiche un formulaire HTML avec champ "nom" et bouton submit
2. Traite la requête POST et affiche "Bonjour [nom]"
3. Utilise une session pour compter le nombre de visites

```
1 import java.io.*;
2 import javax.servlet.*;
3 import javax.servlet.http.*;
4 import javax.servlet.annotation.WebServlet;
5
6 @WebServlet("/bonjour")
7 public class BonjourServlet extends HttpServlet {
8
9     @Override
10    protected void doGet(HttpServletRequest req, HttpServletResponse
resp)
11        throws ServletException, IOException {
12        // Afficher le formulaire
13        resp.setContentType("text/html");
14        PrintWriter out = resp.getWriter();
15        out.println("<html><body>");
16        out.println("<form method='post' action='bonjour'>");
17        out.println("Nom: <input type='text' name='nom'>");
18        out.println("<input type='submit' value='Dire Bonjour'>");
19        out.println("</form>");
20        out.println("</body></html>");
21    }
22
23    @Override
24    protected void doPost(HttpServletRequest req, HttpServletResponse
resp)
```

```

25         throws ServletException, IOException {
26     String nom = req.getParameter("nom");
27     HttpSession session = req.getSession();
28     Integer compteur = (Integer) session.getAttribute("compteur");
29     if (compteur == null) compteur = 1;
30     else compteur++;
31     session.setAttribute("compteur", compteur);
32
33     resp.setContentType("text/html");
34     PrintWriter out = resp.getWriter();
35     out.println("<html><body>");
36     out.println("<h1>Bonjour " + nom + " !</h1>");
37     out.println("<p>Vous avez visite cette page " + compteur + "
fois.</p>");
38     out.println("<a href='bonjour'>Retour</a>");
39     out.println("</body></html>");
40 }
41 }

```

11.2 Exercice 2 : Configuration web.xml

Exercice type

Configurez une Servlet "LoginServlet" du package "web" avec les URLs "/login" et "/connexion".

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
5         http://xmlns.jcp.org/xml/ns/javaee/web-
app_4_0.xsd"
6     version="4.0">
7
8     <display-name>MonApplication</display-name>
9
10    <!-- Declaration de la Servlet -->
11    <servlet>
12        <servlet-name>LoginServlet</servlet-name>
13        <servlet-class>web.LoginServlet</servlet-class>
14    </servlet>
15
16    <!-- Mapping des URLs -->
17    <servlet-mapping>
18        <servlet-name>LoginServlet</servlet-name>
19        <url-pattern>/login</url-pattern>
20    </servlet-mapping>
21
22    <servlet-mapping>

```

```
23     <servlet-name>LoginServlet</servlet-name>
24     <url-pattern>/connexion</url-pattern>
25 </servlet-mapping>
26
27 <!-- Page d'accueil par défaut -->
28 <welcome-file-list>
29     <welcome-file>index.html</welcome-file>
30     <welcome-file>index.jsp</welcome-file>
31 </welcome-file-list>
32
33 </web-app>
```


Chapitre 12

Synthèse - Ce qu'il faut absolument retenir

12.1 Tableau récapitulatif des différences essentielles

Comparaison	Point clé
GET vs POST	GET=URL / POST=corps
POST vs PUT	POST=créer / PUT=remplacer totalement
forward() vs sendRedirect()	forward=serveur (même requête) / sendRedirect=client (nouvelle requête)
Statement vs PreparedStatement	Statement=SQL concaténé, vulnérable / PreparedStatement=paramétré, sécurisé
JPA vs JDBC	JPA=ORM haut niveau / JDBC=bas niveau, SQL manuel
JPA vs Hibernate	JPA=spécification / Hibernate=implémentation
Servlet vs JSP	Servlet=contrôleur / JSP=vue (HTML+Java)

TABLE 12.1 – Tableau de révision rapide

12.2 Annotations à connaître par cœur

Annotation	Technologie	Description
@WebServlet	Servlet	Déclare une Servlet avec son mapping URL
@Entity	JPA	Déclare une classe comme entité persistante
@Id	JPA	Définit la clé primaire
@GeneratedValue	JPA	Stratégie de génération automatique des IDs
@ManyToOne	JPA	Relation plusieurs-à-un
@OneToMany	JPA	Relation un-à-plusieurs (avec mappedBy)
@Inheritance	JPA	Définit la stratégie d'héritage
@SpringBootApplication	Spring Boot	Lancement application Spring Boot
@RestController	Spring MVC	Contrôleur REST
@Autowired	Spring	Injection automatique de dépendances
@Service	Spring	Marque une classe comme service métier
@Repository	Spring	Marque une classe comme DAO
@Transactional	Spring	Délimite une transaction
@Query	Spring Data	Définit une requête personnalisée

TABLE 12.2 – Annotations essentielles

12.3 Checklist de révision finale

Checklist - Ce que vous devez maîtriser avant l'examen

1. Définir Java EE et le rôle du conteneur web
2. Expliquer le cycle de vie d'une Servlet (init/service/destroy)
3. Différence GET/POST/PUT
4. Configurer une Servlet (annotation vs web.xml)
5. Différence forward() vs sendRedirect()
6. Différence JPA vs JDBC vs Hibernate
7. Mapper l'héritage JPA (stratégie JOINED)
8. Mapper les relations JPA (@ManyToOne, @OneToMany)
9. Dessiner l'architecture MVC à 3 couches
10. Citer les Spring Boot Starters
11. Expliquer @Autowired et @Query
12. Connaître les codes HTTP (200, 404, 405, 500)

BONNE CHANCE POUR L'EXAMEN !